Understanding Buildpacks

A Deep Dive Into Their Functionality

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Abstract

Understanding Buildpacks: Delving Deep into Their Functionality

Cloud Native Buildpacks make it easier for developers to create container images, offering a seamless experience with built-in features such as rebasing, reproducibility, and support for multiple entry points. In this talk, we'll start by covering the basics of Buildpacks, looking at their architecture and the benefits they bring compared to traditional methods of building container images.

We'll then explore the lifecycle component, which is the central binary of Cloud Native Buildpacks. This part will reveal how it manages the conversion of source code into OCI images and oversees the various stages of the build process, including detection, installation, and export.

We will also discuss the Paketo Buildpacks project, emphasizing its flexibility and community focus. This section will show how Paketo improves the developer experience by offering a curated selection of buildpacks that make configuration easier and enhance performance. Additionally, we'll share real-world examples of how organizations are using Buildpacks to streamline their CI/CD pipelines and speed up their deployment cycles. To wrap up, we'll go over some best practices for integrating Buildpacks into your workflows, providing tips on maximizing their features for better performance and security.

Understanding Buildpacks:

Delving Deep into Their Functionality

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

Show of Hands ...

- Who has more than 10 Dockerfiles in their organisation?
- Keep your hand up if they all look the same...
- Everything I claim in this talk, I'll prove live.

Who am I?

Patrick Baumgartner

Technical Agile Coach @ **42talents**

My focus is on the **development of software solutions *with* humans**.

Coaching, Architecture, Development, Reviews, and Training.

Lecturer @ **Zurich University of Applied Sciences ZHAW**

Co-Organizer of **Voxxed Days Zurich**, **JUG Switzerland**, Java Champion, Oracle ACE Pro Java ...

Agenda

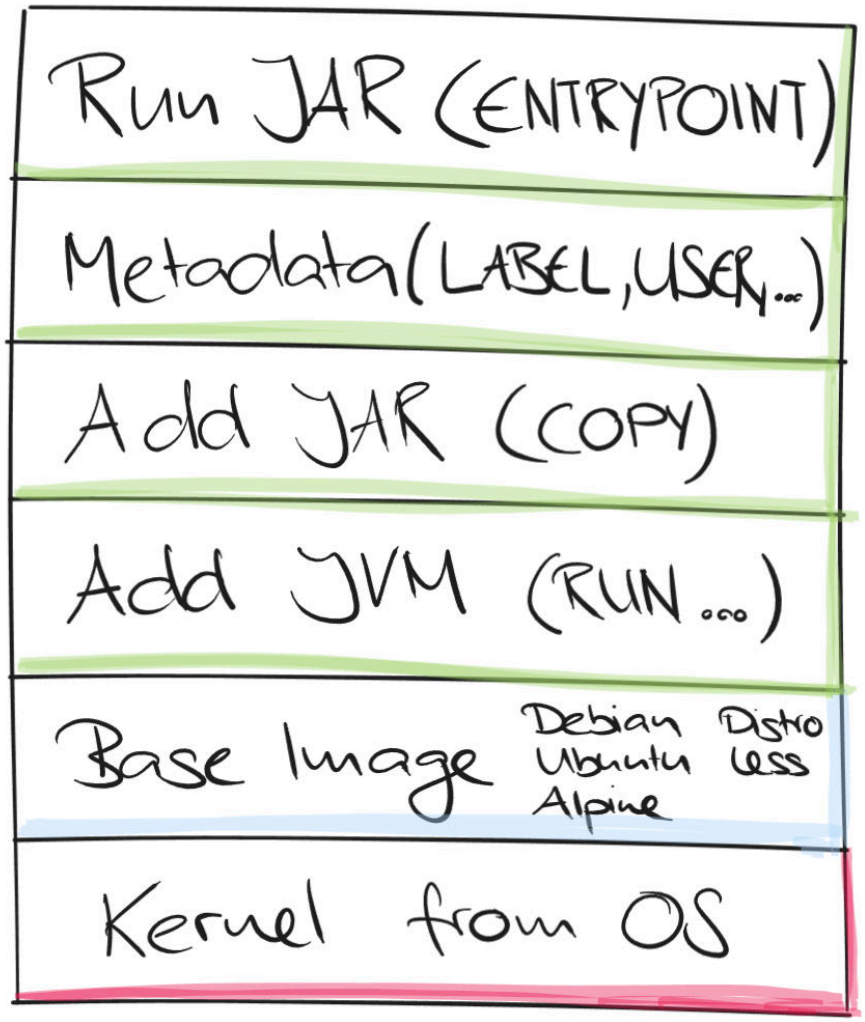
Agenda

1. **The fundamentals:** containers and Cloud Native Buildpacks
2. **The case for switching:** Buildpacks vs. Dockerfiles
3. **Under the hood:** lifecycle, builders, and customization
4. **Java superpowers:** SBOM, observability, jlink, AOT cache
5. **Proof:** live demo, war stories, and CI/CD integration

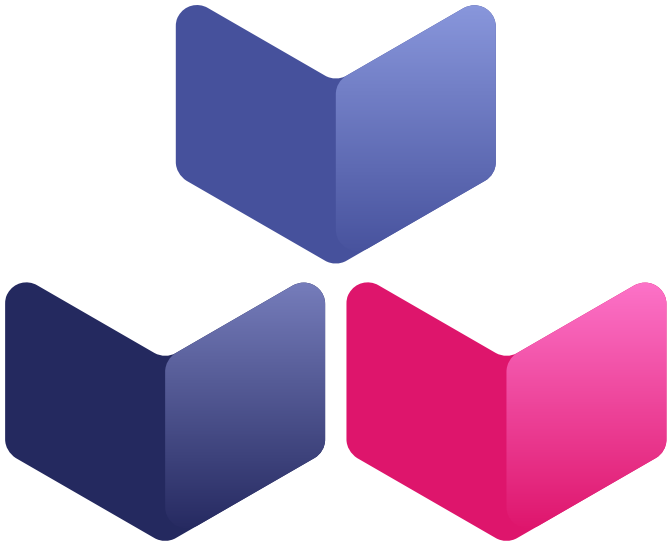
What Are Containers, Really?

What Are Containers, Really?

- Isolated processes using Linux primitives: `chroot`, `namespaces`, and `cgroups`
- Provide filesystem, resource, and process isolation
- Lightweight, fast to start, and portable
- Run on any host with a container runtime like Docker or containerd



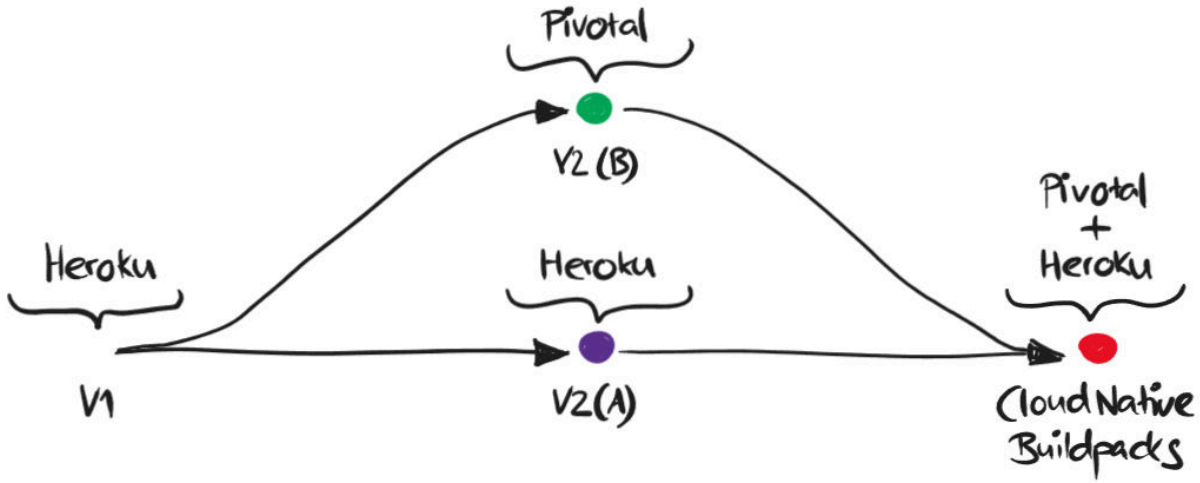
What Are Cloud Native Buildpacks?



Buildpacks.io

What Are Cloud Native Buildpacks?

- Convert source code into OCI-compliant images
- Automatically detect app type and required dependencies
- No Dockerfile needed
- Originated at Heroku, now under CNCF
- Co-developed by Pivotal/VMware and Heroku



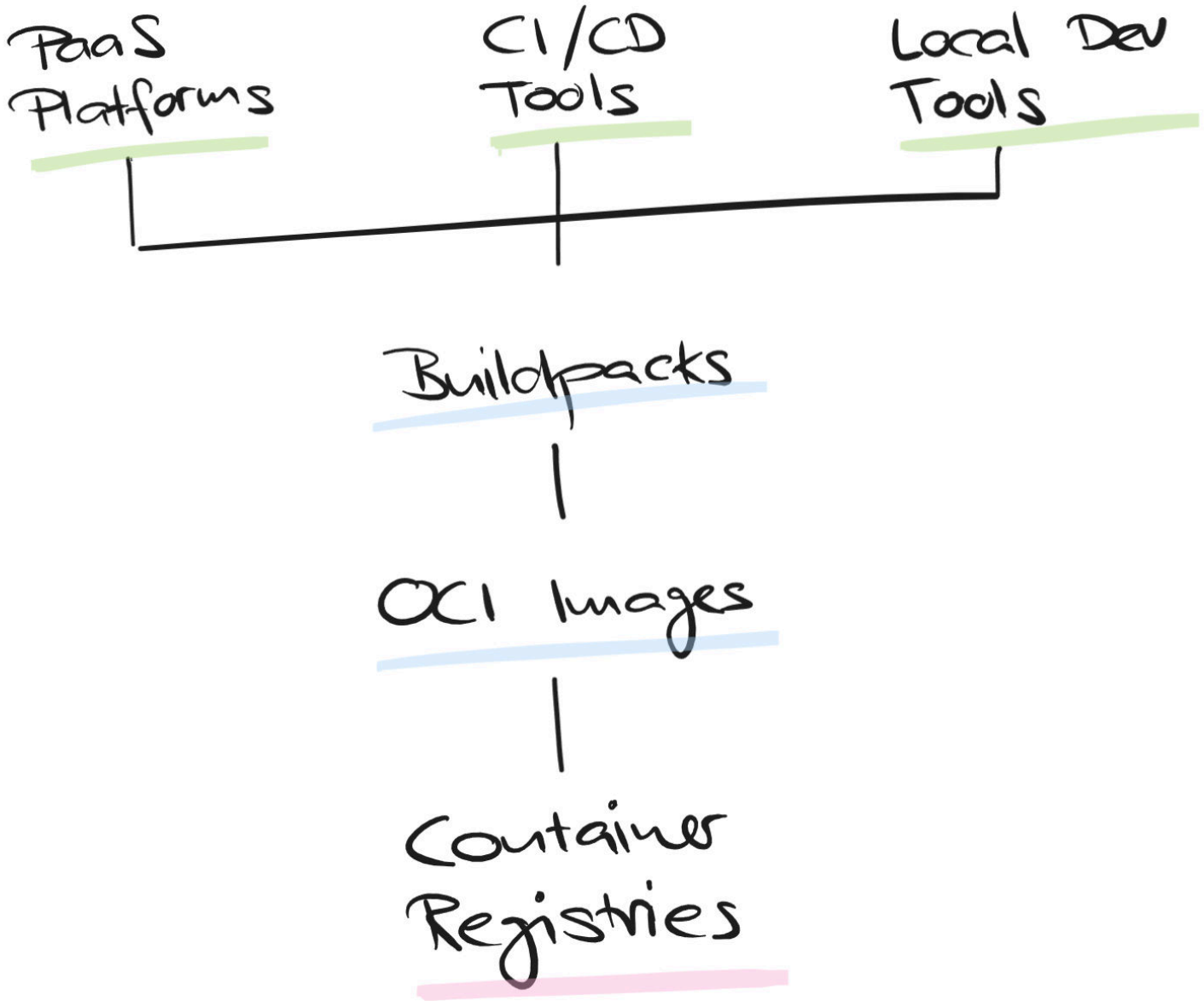
paketo

buildpacks

Your app,
in your favorite language,
ready to run in the cloud

Why Cloud Native Buildpacks Are Relevant Today

- Simplifies building container images without Dockerfiles
- Ensures consistent builds across teams and environments
- Enhances security via curated and updatable buildpacks
- Accelerates CI/CD with reproducible builds
- Integrates smoothly into modern DevOps workflows


PaaS
Platforms
CI/CD
Tools
Local Dev
Tools
Buildpacks
OCI Images
Container
Registries

Widespread Ecosystem Adoption

- **PaaS Platforms:** Heroku, Fly.io, Cloud Foundry, Google Cloud Run, DigitalOcean, Azure Container Apps, Epinio, etc.
- **CI/CD Tools:** GitHub Actions, CircleCI, GitLab, Jenkins, Tekton, etc.
- **Local Dev Tools:** Spring Boot, Quarkus, Pack CLI, Maven/Gradle, kpack, Skaffold, etc.
- **Container Registries:** Docker Hub, GCR, ECR, etc. (OCI-compliant images)

From the Field: A Platform Team's Story

Before: 40+ services, each with its own Dockerfile

- 6 different base images, 4 JDK vendors, no consistent memory tuning
- A critical OS CVE meant **weeks** of coordinated rebuilds and re-tests

After: one curated builder, owned by the platform team

- `pack build` / `./mvnw spring-boot:build-image`
  - everywhere, same result
- CVE patch day: `pack rebase` across the registry
  - in **hours instead of weeks**
- SBOMs generated on every build, ready before the auditors ask

* Composite from real consulting engagements. Numbers vary by team; the pattern repeats.

> From source code to production-ready container image, standardized and Dockerfile-free. Buildpacks automate and unify application delivery.

Cloud Native Buildpacks

475 followers https://buildpacks.io

Pinned

pack Public
CLI for building apps using Cloud Native Buildpacks
Go 2.7k 303

spec Public
Specification for Cloud Native Buildpacks
Shell 259 71

lifecycle Public
Reference implementation of the Cloud Native Buildpacks lifecycle
Go 194 115

rfcs Public
RFCs for Cloud Native Buildpacks
Shell 58 74

community Public
Community content for the Cloud Native Buildpacks (CNB) project
47 24

samples Public
Samples for Cloud Native Buildpacks
Shell 208 151

Why Choose Buildpacks Instead of Dockerfiles?

The Dockerfile Tax

"Your Dockerfile might be fine. But do you trust the 47 other ones in your mono-repo?"

- N teams × 2–4 services each = Dockerfile sprawl
- Every team picks a different base image, JDK vendor, and OS
- One critical CVE → rebuild, retest, redeploy **every single service**
- No consistency in memory settings, user, or signal handling
- Knowledge is locked in tribal lore, not in code

This is an organisational problem at scale, not a technical one.

The Limitations of Dockerfiles

This is what a *senior* developer writes:

```
FROM eclipse-temurin:25-jdk-noble AS build
WORKDIR /app
COPY . .
RUN ./mvnw -q package -DskipTests

FROM eclipse-temurin:25-jre-noble
RUN addgroup --system app && adduser --system --ingroup app app
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
USER app
EXPOSE 8080
HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Still missing: memory calculator, CA certificate injection, SBOM, reproducible layers, signal handling (`tini`), JVM ergonomics for containers, automatic base-image patching.

Reproducibility: Dockerfiles vs. Buildpacks

Dockerfiles are non-deterministic by design:

```
RUN apt-get install -y curl   # resolves differently on different days
FROM ubuntu:latest            # "latest" drifts silently
```

- Two builds from identical source → **different image digests**
- Breaks audit trails, supply chain attestations, and rollback guarantees

Buildpacks produce reproducible images:

- Same source + same builder = **identical OCI digest** every time
- Layers are content-addressed and cached precisely
- Builder version is explicit and pinnable:

```
pack build my-app \
  --builder paketobuildpacks/builder-noble-java-tiny:0.0.180
```

- Works with **SLSA** provenance and **Sigstore/cosign** signing out of the box

Image Size & CVE Reality Check

Approach	Compressed size	CVEs (Trivy)
`FROM openjdk` (naive)	~650 MB	200+
Multi-stage + `eclipse-temurin:jre`	~250 MB	40–80
Paketo `builder-noble-java-tiny`	~180 MB	< 10
Distroless + jlink (custom builder)	~80–140 MB	0 (no shell, no pkgmgr)

- **Smaller = faster pull, faster start, smaller blast radius**
- Distroless: no shell means attackers can't `exec` in even if they break the app
- Paketo rebuilds the run image nightly and patches CVEs **without a code change**

```
# Scan it yourself
trivy image my-app
```

The Buildpacks CLI: Simplicity in Action

```
pack builder suggest
```

Google Cloud Buildpacks:

```
pack build my-app --builder gcr.io/buildpacks/builder
```

Paketo Buildpacks:

```
pack build my-app --builder paketobuildpacks/ubuntu-noble-builder # Ubuntu 24.04 LTS, all languages
pack build my-app --builder paketobuildpacks/builder-noble-java-tiny # Ubuntu 24.04 LTS, Java-only
pack build my-app --builder paketobuildpacks/ubuntu-resolute-builder # Ubuntu 26.04 LTS, newest
```

Spring Boot Maven Plugin:

```
./mvnw spring-boot:build-image
```

Inspect a built image:

```
pack inspect my-app   # shows builder, buildpacks, stack, and run-image
```

```
Calculating JVM memory based on 12395640K available memory
For more information on this calculation, see https://paketo.io/docs/reference/java-reference/#memory-calculator
Calculated JVM Memory Configuration: -XX:MaxDirectMemorySize=10M -Xmx11758165K -XX:MaxMetaspaceSize=125474K -XX:ReservedCodeCacheSize=240M -Xss1M   ⬅ (1) memory calculator
  (Total Memory: 12395640K, Thread Count: 250, Loaded Class Count: 19739, Headroom: 0%)
Enabling Java Native Memory Tracking
Adding 121 container CA certificates to JVM truststore                                                                                        ⬅ (2) CA certs, no keytool
Spring Cloud Bindings Enabled
Picked up JAVA_TOOL_OPTIONS: -Djava.security.properties=/layers/paketo-buildpacks_bellsoft-liberica/java-security-properties/java-security.properties
 -XX:+ExitOnOutOfMemoryError -XX:AOTCache=/layers/patbaumgartner_aot-cache/aot-cache/app.aot                                                  ⬅ (3) AOT cache wired in
 -XX:MaxDirectMemorySize=10M -Xmx11758165K -XX:MaxMetaspaceSize=125474K -XX:ReservedCodeCacheSize=240M -Xss1M -XX:+UnlockDiagnosticVMOptions
 -XX:NativeMemoryTracking=summary -XX:+PrintNMTStatistics -Dorg.springframework.cloud.bindings.boot.enable=true

              |\      _,,,--,,_
             /,`.-'`'   ._  \-;;,_
  _______ __|,4-  ) )_   .;.(__`'-'__     ___ __    _ ___ _______
 |       | '---''(_/._)-'(_\_)   |   |   |   |  |  | |   |       |
 |    _  |    ___|_     _|       |   |   |   |   |_| |   |       | __ _ _
 |   |_| |   |___  |   | |       |   |   |   |       |   |       | \ \ \ \
 |    ___|    ___| |   | |      _|   |___|   |  _    |   |      _|  \ \ \ \
 |   |   |   |___  |   | |     |_|       |   | | |   |   |     |_    ) ) ) )
 |___|   |_______| |___| |_______|_______|___|_|  |__|___|_____|  / / / /
 ==========================================================================/_/_/_/

:: Built with Spring Boot :: 4.1.1

2026-08-31T19:06:13.749Z  INFO 1 --- [           main] o.s.s.petclinic.PetClinicApplication     : Starting PetClinicApplication v4.0.0-SNAPSHOT using Java 25.0.4 with PID 1
  (/workspace/BOOT-INF/classes started by cnb in /workspace)
2026-08-31T19:06:13.752Z  INFO 1 --- [           main] o.s.s.petclinic.PetClinicApplication     : No active profile set, falling back to 1 default profile: "default"
2026-08-31T19:06:14.556Z  INFO 1 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-08-31T19:06:14.609Z  INFO 1 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 44 ms. Found 3 JPA repository interfaces.
2026-08-31T19:06:15.171Z  INFO 1 --- [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8080 (http)
2026-08-31T19:06:15.181Z  INFO 1 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-08-31T19:06:15.182Z  INFO 1 --- [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/11.0.24]
2026-08-31T19:06:15.218Z  INFO 1 --- [           main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1400 ms
2026-08-31T19:06:15.394Z  INFO 1 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-08-31T19:06:15.475Z  INFO 1 --- [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection conn0: url=jdbc:h2:mem:b511bf05-2c4d-4eb5-9881-255cf7503e0e user=SA
2026-08-31T19:06:15.476Z  INFO 1 --- [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-08-31T19:06:15.618Z  INFO 1 --- [           main] org.hibernate.orm.jpa                    : HHH008540: Processing PersistenceUnitInfo [name: default]
2026-08-31T19:06:15.657Z  INFO 1 --- [           main] org.hibernate.orm.core                   : HHH000001: Hibernate ORM core version 7.4.5.Final
2026-08-31T19:06:15.931Z  INFO 1 --- [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-08-31T19:06:15.957Z  INFO 1 --- [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
2026-08-31T19:06:17.251Z  INFO 1 --- [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat started on port 8080 (http) with context path '/'
2026-08-31T19:06:17.261Z  INFO 1 --- [           main] o.s.s.petclinic.PetClinicApplication     : Started PetClinicApplication in 3.905 seconds (process running for 4.38)
```

Adding CA Certificates with Dockerfiles

```
FROM openjdk

COPY custom-ca.crt /app
RUN keytool -importcert -file /app/custom-ca.crt
COPY target/*.jar /app/app-runner.jar

WORKDIR /app

EXPOSE 8080

ENTRYPOINT [ "java","-jar" ]

CMD ["app-runner.jar" ]
```

Adding CA Certificates with Buildpacks

```
mkdir -p bindings/ca-certificates
echo "ca-certificates" > bindings/ca-certificates/type
cp my-cert.crt bindings/ca-certificates/my-cert.crt
```

```
bindings/
└── ca-certificates/
    ├── type            # file contains: ca-certificates
    └── my-cert.crt
```

```
pack build my-app \
  --builder paketobuildpacks/ubuntu-noble-builder \
  --volume "$(pwd)/bindings:/platform/bindings"
```

```
BellSoft Liberica JRE 25.0.4: Contributing to layer
  Downloading from https://github.com/bell-sw/Liberica/releases/download/25.0.4+9/bellsoft-jre25.0.4+9-linux-amd64.tar.gz
  Verifying checksum
  Expanding to /layers/paketo-buildpacks_bellsoft-liberica/jre
  Adding 121 container CA certificates to JVM truststore
```

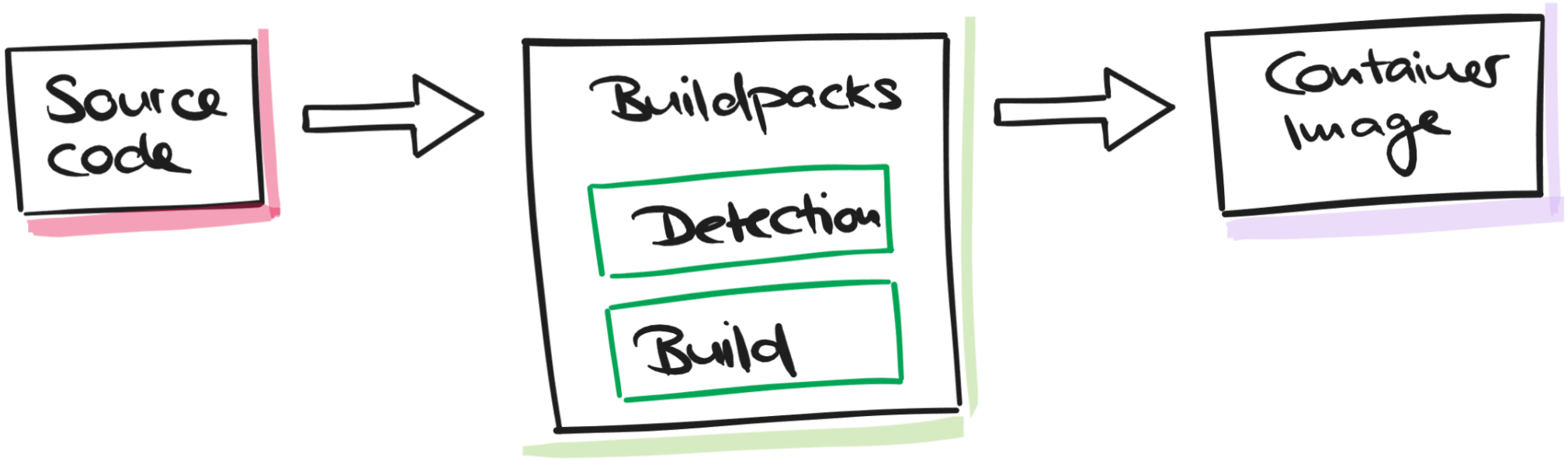
> Imagine if building your Go, Node.js, and Python apps with buildpacks were that simple.

The Basics of Cloud Native Buildpacks

Buildpacks Architecture

- **BuildImage and RunImage**: Base OS (e.g. Ubuntu) → Stack (deprecated)
- **Buildpacks:** Language/runtime logic (Node.js, Java, etc.)
- **Lifecycle:** Orchestrates the build
- **Builder:** All of the above in one image

How Buildpacks Work



- **buildpack.toml:** Provides metadata for buildpacks
- **Detect:** Identify the app type and required dependencies
- **Build:** Install dependencies and create the image

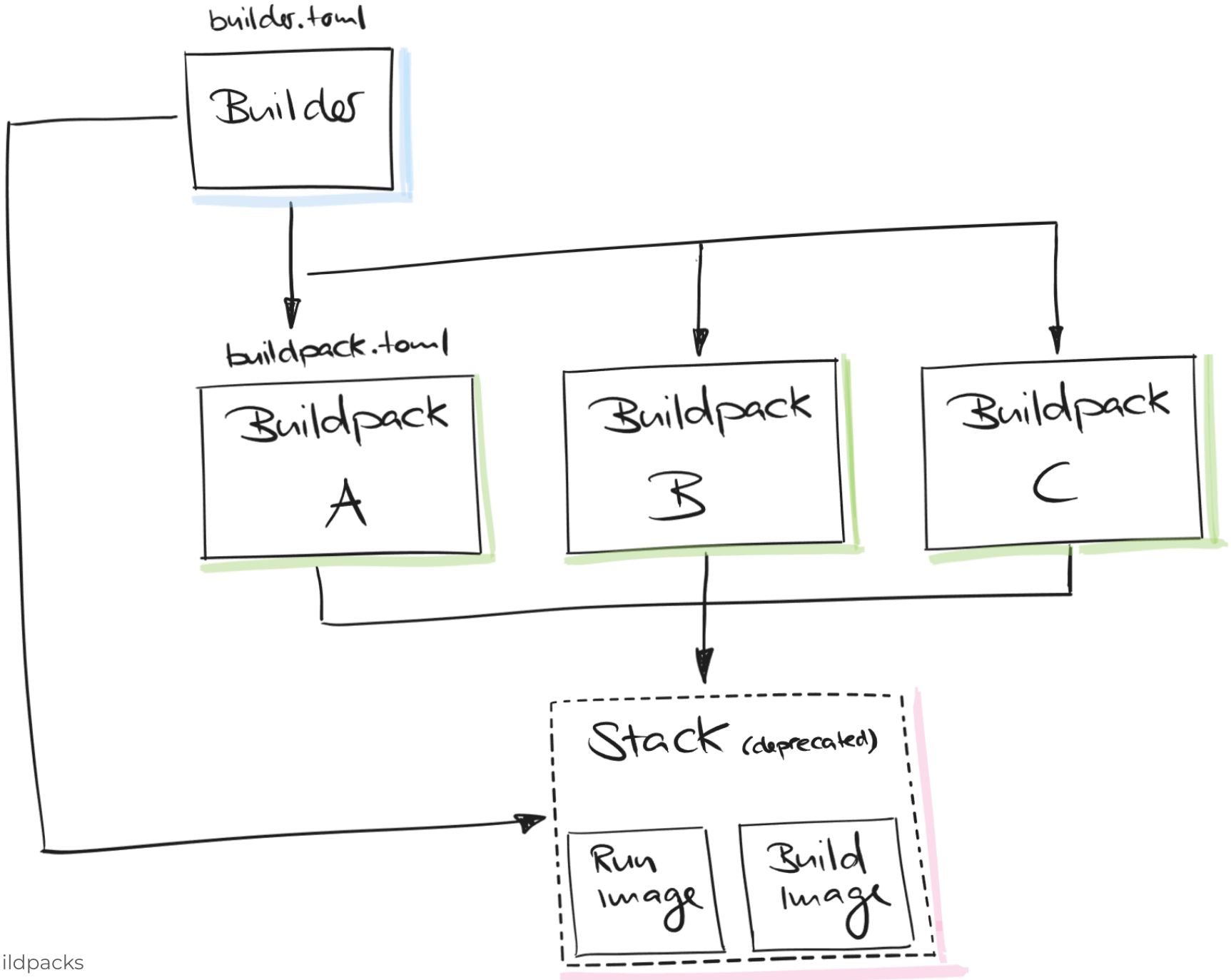
Anatomy of a Buildpack

- `bin/detect`
  - Runs against source to determine if the buildpack is applicable
  - Example:
    - A Java CNB might look for `.java` files or a `.jar`
    - A Node.js CNB might look for `package.json`
- `bin/build`
  - Downloads build-time and run-time dependencies
  - Example:
    - A Java CNB might download a JDK or JRE
    - Compiles sources (if needed)
    - Sets launch command and environment variables

Lifecycle Phases (Create)

The buildpack lifecycle is split into phases. Each plays a critical role in image creation.

- **Analyze:** Provides metadata from previous builds to buildpacks.
- **Detect:** Selects compatible buildpacks and creates a build plan.
- **Restore:** Retrieves cached data from earlier builds.
- **Build:** Rebuilds only the layers that need updating.
- **Export:** Replaces outdated layers with the newly built ones.


builder.toml
Builder
buildpack.toml
Buildpack A
Buildpack B
Buildpack C
Stack (deprecated)
Run Image
Build Image

Customizing and Reusing Buildpacks

- **Why customize?**
  - Select a specific JVM vendor or version (e.g. Adoptium, BellSoft)
  - Add CA certificates for internal services
  - Configure Maven or Gradle to use internal repositories or proxies
  - Extend functionality with custom environment variables or bindings
- **Keep your image updated:**
  - Rebase your image with the latest run image (no rebuild needed)
  - `pack rebase my-app`
- **Browse available buildpacks:** https://registry.buildpacks.io

CVE in the Base OS? Rebase Instead of Rebuilding

A critical vulnerability is patched in Ubuntu. With Dockerfiles:

1. Identify every affected service (good luck)
2. Trigger rebuilds across 50 repositories
3. Re-run full test suites
4. Redeploy everything

CVE in the Base OS? Rebase Instead of Rebuilding

With buildpacks it takes seconds, with no code change:

```
# Swap the run image layer. App layers are untouched.
pack rebase my-app

# Or move to an explicitly newer run image
pack rebase my-app \
  --run-image paketobuildpacks/ubuntu-noble-run-tiny:latest --force
```

- Only the OS layer is replaced; the app layers are **reused as-is**
- The image digest changes, so re-sign and re-attest in the same pipeline step
- Works at scale: one script, every image in your registry

paketo

buildpacks

Paketo Buildpacks Overview

- **Available in 4 stack variants:**
  - **Full** – Large set of OS packages for apps with native dependencies
  - **Base** – Core buildpacks for major languages
  - **Tiny** – Minimal footprint, ideal for small apps or functions
  - **Static** – For statically linked binaries (e.g. Go, GraalVM native)
- **Broad language support:**
  - Java, Node.js, Python, Go, .NET, PHP, Ruby, Rust, and more
- **Rich feature set:**
  - Support for custom CA certificates
  - SBOM generation using Syft
  - OCI image annotations and additional labels
  - Arm64 and Red Hat UBI compatibility

Software Bill of Materials (SBOM) Support

```
pack sbom download my-app --output-dir ./my-app-sbom
```

```
cat ./my-app-sbom/layers/sbom/launch/paketo-buildpacks_executable-jar/sbom.cdx.json | \
  jq '.components[] | select(.name=="jackson-databind")'
```


```
{
  "bom-ref": "pkg:maven/tools.jackson.core/jackson-databind@3.1.5?package-id=d4471ad5673ddc27",
  "cpe": "cpe:2.3:a:jackson-databind:jackson-databind:3.1.5:*:*:*:*:*:*:*",
  "group": "tools.jackson.core",
  "name": "jackson-databind",
  "version": "3.1.5",
  "purl": "pkg:maven/tools.jackson.core/jackson-databind@3.1.5",
  "properties": [
    { "name": "syft:package:type", "value": "java-archive" },
    { "name": "syft:package:language", "value": "java" }
  ]
}
```

Observability Without Code Changes

Paketo Java buildpacks inject observability tooling at **build time**, with no library dependencies in your code.

JMX exposes MBeans to any JMX-compatible tool:

```
docker run -p 8080:8080 -p 5000:5000 \
  -e BPL_JMX_ENABLED=true \
  -e BPL_JMX_PORT=5000 \
  my-app
```

Spring Boot Actuator is wired in automatically when detected on the classpath:

```
curl http://localhost:8080/actuator/health
curl http://localhost:8080/actuator/metrics
```

OpenTelemetry injects the Java agent at build time, with zero code change:

```
pack build my-app --env BP_OPENTELEMETRY_ENABLED=true

docker run -p 8080:8080 -e OTEL_EXPORTER_OTLP_ENDPOINT=https://otel-collector:4317 my-app
```

Advanced Features in Java Buildpacks

- Set JVM version using `BP_JVM_VERSION`
- Easy Java vendor selection using buildpack order

```
<buildpacks>
  <buildpack>gcr.io/paketo-buildpacks/adoptium</buildpack>
  <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
</buildpacks>
```

- Optimizations for Spring Boot (AOT, CDS, Cloud Bindings) and memory management (memory calculator)
- jlink and JMX support
- Native image support with GraalVM

BP_* and BPL_* Environment Variables

- **`BP_*`** build-time: passed via `--env` or `<env>` in `pom.xml`, configures what the buildpack produces
- **`BPL_*`** launch-time: set at `docker run` or in Kubernetes, tunes runtime behaviour without rebuilding

Variable	Phase	Purpose
`BP_JVM_VERSION`	Build	JDK version to install
`BP_JVM_JLINK_ENABLED`	Build	Create a custom JRE with jlink
`BP_SPRING_AOT_ENABLED`	Build	Enable Spring AOT processing
`BPL_JMX_ENABLED`	Launch	Enable JMX
`BPL_DEBUG_ENABLED`	Launch	Enable remote debugger (JDWP)
`BPL_DEBUG_PORT`	Launch	Debugger port (default `8000`)

```
# Remote debug a running container
docker run -p 8080:8080 -p 8000:8000 -e BPL_DEBUG_ENABLED=true -e BPL_DEBUG_PORT=8000   my-app
```

Demo

Challenges & Solutions

Patching the Base Image

- Base images do not contain fonts 😔

```
FROM paketobuildpacks/ubuntu-noble-run:latest

USER root

## Idea from: https://blog.adoptopenjdk.net/2021/01/prerequisites-for-font-support-in-adoptopenjdk/
RUN apt-get update \
  && apt-get install -y --no-install-recommends \
  wget \
  fontconfig \
  && rm -rf /var/lib/apt/lists/*

# Install Noto Emoji Manually
RUN mkdir -p /usr/share/fonts/truetype/noto \
  && cd /usr/share/fonts/truetype/noto \
  && wget https://github.com/googlefonts/noto-emoji/raw/9a5261d871451f9b5183c93483cbd68ed916b1e9/fonts/NotoEmoji-Regular.ttf \
  && fc-cache -fv

USER cnb
```

When to Create a Custom Buildpack

- Custom JDK Version & JVM Tuning
- Application Configuration Injection
- Metrics/Monitoring Integration
- License Compliance
- Logging Configuration

```
pack buildpack new io.example/my-buildpack \
  --path ./my-buildpack

pack build my-app \
  --buildpack ./my-buildpack \
  --builder paketobuildpacks/ubuntu-noble-builder
```

Custom Builder

- Using a custom builder for a custom base-image and run-image (stack)
- Allows a (pre-)selection of buildpacks to be used

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <builder>patbaumgartner/buildpack-builder:noble</builder>
    </image>
  </configuration>
</plugin>
```

Distroless Builder for Java

- Build stack: **Ubuntu 24.04** – full toolchain for compiling Java apps
- Run stack: **`gcr.io/distroless/cc:nonroot`** – no shell, no package manager, non-root
- Supports: **JVM** and **GraalVM Native Image**
- Images published to GHCR with **SLSA provenance** attestations

```
<!-- pom.xml – Spring Boot Maven Plugin -->
<image>
  <builder>ghcr.io/patbaumgartner/distroless-buildpack-builder-java-tiny:latest</builder>
  <env>
    <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
  </env>
</image>
```

See: https://github.com/patbaumgartner/distroless-buildpack-builder-java-tiny

Custom Java Buildpacks

```
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <configuration>
    <image>
      <buildpacks>
        <!-- <buildpack>gcr.io/paketo-buildpacks/adoptium:latest</buildpack> -->
        <!-- <buildpack>gcr.io/paketo-buildpacks/alibaba-dragonwell:latest</buildpack> -->
        <!-- <buildpack>gcr.io/paketo-buildpacks/amazon-corretto:latest</buildpack> -->
        <!-- <buildpack>gcr.io/paketo-buildpacks/azul-zulu:latest</buildpack> -->
        <!-- <buildpack>gcr.io/paketo-buildpacks/bellsoft-liberica:latest</buildpack> -->
        <!-- <buildpack>gcr.io/paketo-buildpacks/eclipse-openj9:latest</buildpack> -->
        <!-- <buildpack>gcr.io/paketo-buildpacks/graalvm:latest</buildpack> -->
        <!-- <buildpack>gcr.io/paketo-buildpacks/sap-machine:latest</buildpack> -->
        <buildpack>gcr.io/paketo-buildpacks/oracle:latest</buildpack>
        <!-- Used to inherit all the other buildpacks -->
        <buildpack>urn:cnb:builder:paketo-buildpacks/java</buildpack>
      </buildpacks>
    </image>
  </configuration>
</plugin>
```

JLink

Same plugin block as before. **Only the <env> changes:**

```
<image>
  <buildpacks>
    <buildpack>gcr.io/paketo-buildpacks/java:latest</buildpack>
  </buildpacks>
  <env>
    <!-- build a right-sized JRE instead of shipping the full JDK -->
    <BP_JVM_JLINK_ENABLED>true</BP_JVM_JLINK_ENABLED>
    <BP_JVM_JLINK_ARGS>--add-modules
      java.base,java.compiler,java.desktop,java.instrument,java.net.http,java.prefs,java.rmi,
      java.scripting,java.security.jgss,java.sql.rowset,jdk.jfr,jdk.management,jdk.net,jdk.unsupported
      --compress=zip-6 --no-header-files --no-man-pages --strip-debug</BP_JVM_JLINK_ARGS>
  </env>
</image>
```

AOT and App CDS (< Java 25)

Again, **only the `<env>` changes** (plus the `process-aot` execution):

```
<image>
  <buildpacks>
    <buildpack>gcr.io/paketo-buildpacks/java:latest</buildpack>
  </buildpacks>
  <env>
    <!-- CDS training run + Spring AOT at build time -->
    <BP_JVM_CDS_ENABLED>true</BP_JVM_CDS_ENABLED>
    <BP_SPRING_AOT_ENABLED>true</BP_SPRING_AOT_ENABLED>
  </env>
</image>
```

```
<executions>
  <execution>
    <id>process-aot</id>
    <goals>
      <goal>process-aot</goal>
    </goals>
  </execution>
</executions>
```

AOT and AOT Caching (>= Java 25)

On Java 25, CDS is superseded by the **AOT cache** (JEP 483/514). **Only the `<env>` changes:**

```
<image>
  <buildpacks>
    <buildpack>gcr.io/paketo-buildpacks/java:latest</buildpack>
  </buildpacks>
  <env>
    <!-- training run happens inside the build, cache baked into a layer -->
    <BP_SPRING_AOT_ENABLED>true</BP_SPRING_AOT_ENABLED>
    <BP_JVM_AOTCACHE_ENABLED>true</BP_JVM_AOTCACHE_ENABLED>
    <TRAINING_RUN_JAVA_TOOL_OPTIONS>-Dtelegrambots.enabled=false</TRAINING_RUN_JAVA_TOOL_OPTIONS>
  </env>
</image>
```

- Keep the `process-aot` execution from the previous slide
- The launcher appends `-XX:AOTCache=...` automatically at runtime

AOT Cache: The Measured Numbers

Same Spring Petclinic image, same machine, only the AOT cache toggled:

Startup (median of 3)	**`Started PetClinicApplication in`**
AOT cache **off** (`-XX:AOTMode=off`)	**5.7 s**
AOT cache **on** (default)	**3.9 s**

- **~30% faster startup** from one build-time flag, no code change
- The cache (~120 MB) ships as its own OCI layer and is rebuilt only when the app changes
- Measure it yourself:

```
docker run --rm my-petclinic-aot                                  # cache on
docker run --rm -e JAVA_TOOL_OPTIONS='-XX:AOTMode=off' my-petclinic-aot  # cache off
```

Integrating Buildpacks into CI/CD Pipelines

- **Choose the Right Buildpacks**
  - Use only officially maintained and security-audited buildpacks.
- **Automate Security Scans**
  - Integrate static and dynamic security scans immediately after the build process.
- **Optimize Caching and Layer Reuse**
  - Enable buildpack-specific caching to avoid redundant steps.
- **Ensure Versioning and Traceability**
  - Explicitly document and version the buildpacks used in your CI configuration.
- **Integrate Observability and Metrics**
  - Continuously monitor build durations, error rates, and artifact sizes.

Summary and Key Takeaways

Why Use Cloud Native Buildpacks?

- No Dockerfile needed
- Focus on providing value to the end users
- Composable (Node.js, Bash, Java, Python, ...)
- Build any app with the same set of buildpacks
- Only rebuild layers that have changed
- Rebasing without rebuilding (put your app on a new base layer)

Q&A

I ❤ Feedback 🙏

Understanding Buildpacks:

Delving Deep into Their Functionality

Patrick Baumgartner
42talents GmbH, Zürich, Switzerland

@patbaumgartner
patrick.baumgartner@42talents.com

	Cloud Native Buildpacks	Dockerfile	source-to-image (s2i)	Jib	ko
Advanced Caching	Yes	No	Yes	No	Yes
Auto-detection	Yes	No	Yes	Yes	Yes
Bill-of-Materials	Yes	No	No	No	Yes
Modular / Pluggable	Yes	No	No	N/A †	N/A †
Multi-language	Yes	Yes	Yes	No	No
Multi-process	Yes	No	No	No	No
Minimal app image	Yes	Yes ♦	Yes ‡	Yes	Yes
Rebasing	Yes	No	No	No	No
Reproducibility	Yes	No	No	Yes	Yes
Reusability	Yes	No	Yes	N/A †	N/A †

Resources: Official Documentation and Tools

- Cloud Native Buildpacks (CNCF)
- Paketo Buildpacks
- Buildpacks Registry
- Spring Boot Buildpacks Integration
- Pack CLI Docs
- GraalVM Native Image Support
- Syft – SBOM Generation
- Snyk Docker Best Practices
- Datadog Java Integration
- What has changed on how we ship the Builders and the base images